

ESILAB

AI Project Guide

Architecture, data model, main functions, API contracts, features, and safe extension guidance for another AI assistant.

3 applications

Public site, admin console, backend API

Core capability

Multi-supplier catalogue with Excel import

Generated: 27 July 2026 **Purpose:** a compact technical handoff for an AI agent that must understand, extend, test, or deploy EsiLab.

1. System purpose

EsiLab is a multi-supplier laboratory-equipment catalogue. It intentionally separates a public, read-only catalogue from an authenticated administration application.

The system lets administrators:

- create suppliers and define how each supplier's Excel columns map to the canonical product fields;
- import .xlsx/.xls supplier files, create or update products by supplier + SKU, and inspect import logs;
- manually create, edit, enable/disable, and delete catalogue products;
- manage pending user registrations and approved admin/employer accounts.

Visitors can browse, search, paginate through, and view product records. A product-detail assistant can use OpenRouter when configured and falls back to deterministic answers based only on the displayed product data.

2. Repository map

Directory	Role	Runtime
backend2/	REST API, PostgreSQL access, Excel ingestion, authentication	Node.js / Express, port 3000 + 3001
lab-site/	Public marketing site and catalogue	Next.js 14 / React 18, port 4001
lab-admin2/	Protected catalogue-management console	Next.js 14 / React 18, port 4000
RAPPORT_AVANCEMENT_SITE_WEB.md	Earlier project-status report; useful context, but verify it against current code	Markdown

The root has no workspace-level package manifest. Install, run, build, and test each application independently.

3. Architecture and request flow

Public visitor

- > lab-site (Next.js :4001)
- > public API (Express :3000, /api)
- > PostgreSQL

Administrator

- > lab-admin2 (Next.js :4000)
- > Next proxy (/api/admin/*)
- > admin API (Express :3001, /admin)
- > PostgreSQL

Supplier Excel file

- > multipart upload to admin API
- > ImportService + XLSX parser
- > Product upsert by supplier_id + JSON data.sku
- > import_logs audit record

There are two Express entry points in backend2/src/:

- server.js starts the public, GET-only catalogue API on PORT (default 3000). It enables Helmet, permissive GET CORS, a 300-requests/15-minute limiter, static /assets, health check, public routes, and central error handling.

- `adminServer.js` starts the admin API on `ADMIN_PORT` (default 3001). It permits configured `ADMIN_CORS_ORIGINS`, enables Helmet, a 200-requests/15-minute limiter, health check, admin routes, and central error handling.

4. Technology choices

- **Backend:** Node.js, Express, Knex, pg, Joi, Multer, SheetJS/XLSX, slugify, Winston, Helmet, CORS, and express-rate-limit.
- **Database:** PostgreSQL. Product attributes live in flexible `jsonb` rather than a fixed product-column schema.
- **Authentication:** custom HMAC-SHA256 JWT-like bearer token, PBKDF2 password hashes, user approval statuses, and role checks for account-management routes.
- **Public/Admin UI:** Next.js 14 App Router, React 18, TypeScript, Tailwind CSS, Lucide icons. Admin also uses Recharts and a small built-in English/French dictionary.
- **Optional AI:** server-side call from the public site to OpenRouter's chat-completions API.

5. Data model

Core tables

Table	Key fields and responsibility
suppliers	supplier identity, slug, contacts, <code>header_row_number</code> , active flag. The header row determines which Excel rows are skipped.
product_fields	canonical field catalogue such as <code>sku</code> , <code>name</code> , <code>description</code> , <code>image_url</code> , <code>brand</code> , <code>datasheet_url</code> ; also labels, display type, required flag, and order.
supplier_field_mappings	maps each supplier's Excel column letter (such as A or AA) to a canonical field. One mapping per supplier/field.
products	<code>supplier_id</code> , public slug, <code>data_jsonb</code> , active flag, timestamps. Data must include SKU and name for manual creation. Prices are intentionally stripped.
categories	optional hierarchy through <code>parent_id</code> .
product_categories	product/category many-to-many pivot.
import_logs	supplier import lifecycle, filename, row counters, JSON errors, start/completion timestamps.
users	name, email, PBKDF2 hash, admin/employer role, pending/approved/rejected status, approval time.

Important design rule

`products.data` is the source of product detail. The model deserializes this JSON before returning it. Search uses JSON expressions for SKU, name, descriptions, brand, and category. SKU is additionally indexed and defines upsert identity **within a supplier**.

The database adds GIN and trigram indexes for product JSON searching and descending import-log indexes for the administration views.

6. Backend modules and important functions

Public catalogue

- `controllers/public/catalogueController.js`
- `list`: paginated search/filter product list (`page`, `limit`, `search`, `category_id`).
- `show`: finds an active product by slug and enriches its response with `_fieldMeta` so the UI can render dynamic fields.
- `categories` and `categoryProducts`: category discovery and paginated category results.
- `models/Product.js`
- `findAll`: applies active, supplier, category, and text filters; returns `{ data, meta }`.
- `findBySlug`, `findById`, `findBySku`: product lookup variants.
- `create`, `update`, `delete`: management operations.
- `upsertBySku`: transactional row-by-row import writer; updates matching supplier/SKU products or creates a unique slug.
- `_uniqueSlug`: deconflicts slugs in the database.

Supplier configuration

- `models/Supplier.js`: supplier CRUD, slug generation, and `findWithMappings` joined view.
- `models/ProductField.js`: canonical field CRUD/query helper.
- `models/SupplierFieldMapping.js`
- `getMappingMap`: creates $O(1)$ `{ field_key: mapping }` lookup for import parsing.
- `replaceForSupplier`: atomically replaces every mapping for one supplier.
- `controllers/admin/supplierController.js`: Joi validation and HTTP wrappers for supplier/mapping operations.

Excel import workflow

`services/ImportService.js` is the main ingestion service:

1. `colLetterToIndex` converts Excel column labels, including multi-letter labels, to zero-based indexes.
2. `parseExcelBuffer` reads every sheet, skips each sheet's configured header rows, applies mappings, rejects blank/non-product rows, and strips price-like keys (`price`, `cost`, `tarif`, `prix`, and similar).
3. `runImport` records processing state, fetches supplier and mappings, parses rows, invokes `Product.upsertBySku`, persists counts/row errors, and finalizes the import log.

`controllers/admin/importController.js` accepts in-memory multipart `.xlsx/.xls` uploads up to 50 MiB, creates an `import_logs` record, and currently executes `runImport` synchronously. `bull` and `Redis` dependencies exist, but no queue is wired into the active path.

Authentication and authorization

- `authentication/passwords.js`: PBKDF2-SHA256 hashing with per-password random salt and timing-safe verification.
- `authentication/tokens.js`: signs/verifies HMAC-SHA256 tokens that expire after 24 hours. It requires `AUTH_TOKEN_SECRET` or `ADMIN_TOKEN_SECRET`.
- `authentication/authController.js`: public signup creates a pending employer; login only succeeds for approved users and returns bearer token + sanitized profile.
- `authentication/authMiddleware.js`: `requireAuth` validates bearer tokens and user approval; `requireAdmin` checks `req.user.role === 'admin'`.

- `authentication/routes.js`: account approval and all user-management endpoints require both authenticated and admin users.

7. API contract

Public API (port 3000, `/api`)

Method	Route	Result
GET	<code>/products</code>	active products plus pagination metadata; supports <code>page</code> , <code>limit</code> , <code>search</code> , <code>category_id</code>
GET	<code>/products/:slug</code>	active product plus <code>_fieldMeta</code>
GET	<code>/categories</code>	categories
GET	<code>/categories/:slug/products</code>	category plus paginated products
GET	<code>/health</code>	public API health status

Admin API (port 3001, `/admin`)

Authentication is `Authorization: Bearer <token>`. `POST /auth/signup` and `POST /auth/login` are public; all catalogue-management routes run after `requireAuth`.

Area	Routes
Auth	signup, login, current user, pending signups approval/rejection
Users	list/create/show/update/delete users (admin role required)
Dashboard	GET <code>/dashboard/summary</code>
Suppliers	list/create/show/update/delete; get/replace mappings
Imports	upload a supplier file; list supplier logs; show a log
Products	list/show/update/delete; toggle active; list field definitions; manual create

Responses generally wrap entity data in `{ data: ... }`. Product lists use `{ data: [...], meta: { total, page, limit, totalPages } }`. Validation errors use 422, missing records use 404, invalid login uses 401, unapproved login uses 403, and constrained database errors are translated by the global error handler.

8. Public website (`lab-site`)

Pages and features

- `/`: server-rendered landing page with navigation, hero, service panels, catalogue-derived product highlights, and footer.
- `/products`: client-side catalogue with debounced search, category-style UI filters, pagination, loading and error states.
- `/products/[slug]`: product detail, dynamic field display, JSON-LD product data, request-for-quote links (email/Gmail/WhatsApp), and product chat.
- `/solutions`, `/about`, `/contact`: marketing and company information.
- `/api/product-chat`: server route that calls OpenRouter when `OPENROUTER_API_KEY` exists.

- `robots.ts`, `sitemap.ts`, and root metadata: SEO discovery, social metadata, and Organization JSON-LD.

Key frontend utilities

- `lib/catalogue.ts`: server-side home-page catalogue fetch and `toHomeProduct` adapter. It supports `API_BASE_URL` or `NEXT_PUBLIC_API_BASE_URL` but falls back to `localhost`.
- `lib/site-data.ts`: local content and featured-product definitions used by presentation and sitemap code.
- Product detail contains `answerProductQuestion`, a local deterministic fallback. It never invents price, stock, safety, compatibility, installation, or regulatory certainty and redirects those questions to `EsiLab`.
- `ContactForm.tsx` composes a prefilled Gmail compose link. It does **not** persist or send the form to a backend email service.

9. Administration app (lab-admin2)

Page inventory

Dashboard, login/signup, user management, supplier listing/creation/editing, supplier field mappings, supplier Excel import, global import history/detail, product listing/filtering, manual product creation, and product editing are implemented as App Router pages.

Data access and session handling

- `app/api/admin/[...path]/route.ts` is a same-origin proxy. It forwards HTTP method, request headers, query string, body, and response stream to `_${ADMIN_API_URL}/admin/...`
- `lib/api.ts` is the central browser API client. It reads the bearer token from `localStorage`, attaches it to requests, and exposes typed-ish helper functions for the routes.
- `AuthProvider.tsx` restores a saved token through `/auth/me`, stores tokens under `auth_token`, and provides login/logout state.
- `AuthShell.tsx` redirects anonymous users to `/login` and authenticated users away from login/signup; it wraps the sidebar/top bar layout.
- `LanguageProvider.tsx` supplies English/French UI labels and remembers the selected language in the browser.

10. Configuration and operations

Backend environment

Use `backend2/.env.example` as the template. Required operational groups are database (`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`), public port (`PORT`), admin port (`ADMIN_PORT`), CORS origins (`ADMIN_CORS_ORIGINS`), and token signing secret (`AUTH_TOKEN_SECRET`). The initial-admin password must be overridden before first migration.

Frontend environment

- Public site: `NEXT_PUBLIC_SITE_URL`; optional `OPENROUTER_API_KEY` and `OPENROUTER_MODEL`; configure an API base URL for all catalogue callers.
- Admin site: `ADMIN_API_URL`, normally the reachable admin Express URL.

Typical local sequence

1. In `backend2`, configure `.env`, install packages, run migrations, run seed, start public and admin servers.
2. In `lab-admin2`, configure `ADMIN_API_URL`, install packages, start port 4000.
3. In `lab-site`, configure the site/API URLs, install packages, start port 4001.

Backend scripts include `migrate`, `migrate:rollback`, `seed`, `dev`, `public start`, `admin start1`, and `Jest test`. The two Next apps provide `dev`, `build`, and `start` scripts.

11. Important limitations and safe-change checklist

These findings reflect the current code and should be handled before production or when modifying related areas:

1. **Role boundary:** catalogue-management routes are guarded by `requireAuth` but not `requireAdmin`; any approved employer can currently modify suppliers, imports, and products. Apply `requireAdmin` at the catalogue-management router boundary if only admins should manage the catalogue.
2. **Public API URLs:** `lab-site/app/products/page.tsx` and `lab-site/app/products/[slug]/page.tsx` hard-code `http://localhost:3000/api`, while the home helper is configurable. Centralize all public calls on a shared environment-backed base URL before deployment.
3. **Long imports:** import execution is synchronous inside the HTTP request. Use a real queue/status polling before handling large supplier files; `Redis/Bull` are installed but unused.
4. **Contact workflow:** the form opens Gmail only; there is no server delivery, storage, anti-spam, or submission audit trail.
5. **Client token storage:** admin bearer tokens are stored in `localStorage`. Evaluate an `HttpOnly` secure-cookie session model if XSS exposure is a concern.
6. **Initial credentials:** a migration seeds an initial account; production must provide a strong secret and strong initial password, then rotate/remove bootstrap access.
7. **SEO catalogue coverage:** `sitemap.ts` derives product entries from local `featuredProducts`, not the live API, so it does not automatically list the whole imported catalogue.
8. **Category linkage:** categories and a product/category pivot exist, but imports primarily preserve fields in product JSON. Confirm the desired process for creating and assigning category relationships.

When changing the project, preserve these invariants:

- never store or expose supplier prices in `products.data`;
- use supplier + SKU for import identity;
- retain the product-data JSON shape and `_fieldMeta` contract unless both API and frontend are updated;
- keep public product endpoints read-only and active-only;
- run migrations/seeds on a non-production database before deployment;
- validate both Next builds and the backend Jest import tests after touching shared flows.

12. Suggested AI work order

For an AI assigned a new feature or bug:

1. Identify which of the three applications owns the user-visible behavior.
2. Trace from Next page/component to its client helper/proxy, then to Express route, controller, model, and migration.
3. Treat product fields as dynamic JSON data; do not assume a hard-coded database column exists.

4. For imports, first inspect supplier mappings and test parsing with the targeted spreadsheet shape.
5. For authentication changes, distinguish public auth routes, `requireAuth`, and `requireAdmin`; test both approved employer and admin behavior.
6. Keep configuration out of source code and avoid reintroducing localhost URLs in deployable pages.

13. Primary navigation paths

Task	Start here
Public product retrieval	<code>backend2/src/routes/public/index.js -> controllers/public/catalogueController.js -> models/Product.js</code>
Admin product management	<code>backend2/src/routes/admin/index.js -> controllers/admin/productController.js -> models/Product.js</code>
Supplier Excel import	<code>controllers/admin/importController.js -> services/ImportService.js -> models/SupplierFieldMapping.js / models/Product.js</code>
Admin browser requests	<code>lab-admin2/lib/api.ts -> lab-admin2/app/api/admin/[...path]/route.ts</code>
Admin login/session	<code>lab-admin2/components/auth/AuthProvider.tsx -> backend authentication/</code>
Public catalogue UI	<code>lab-site/app/products/page.tsx</code> and <code>lab-site/app/products/[slug]/page.tsx</code>
Home catalogue cards	<code>lab-site/app/page.tsx -> lab-site/lib/catalogue.ts</code>
AI product assistant	<code>lab-site/app/products/[slug]/page.tsx -> lab-site/app/api/product-chat/route.ts</code>